Physics as the Referee

A biology language model built from scratch and trained against a tissue simulator

Nam H. Le

Department of Computer Science and Vermont Complex Systems Institute
University of Vermont, nam.le@uvm.edu

Technical report, October 2026. Not peer reviewed.

Abstract

Biomedical language models are trained on text and graded on text, so a fluent answer and a correct one look the same to every metric in the loop. Bioelectricity is a domain where this need not be the case, because a physics simulator can compute what a tissue would do under a described intervention. We use that simulator as the reward for reinforcement learning and ask what it changes. To keep every training token under our control, we built the entire pipeline from scratch: a byte-pair tokenizer and a 40M-parameter transformer pretrained on 0.99 billion tokens of PubMed cell-biology abstracts, followed by instruction tuning, direct preference optimization and group-relative policy optimization against the BETSE bioelectric tissue simulator. On held-out simulator problems the model reaches a reward of 0.945, higher than the strongest open chat model we ran locally, even when that model was given five worked examples (0.904). A sealed exam placed between the training lattice points then cut the score to 0.539 and showed that the model had mostly learned a lookup table; changing where the training simulations were placed, with the algorithm and budget fixed, restored it to 0.777, and in-context examples to 0.835. In a second track we adapt an open pretrained model to the same corpus and find that domain adaptation reliably trades away dialogue, through mechanisms we can measure: answer length inherited from the post-training data, forgetting under LoRA, and two shortcut cues created by the data mixture. We argue that, at this scale, language, facts and physical consequences should be held in three different places (weights, a citable library, and the simulator), and that exam design decides what a simulator-verified score actually certifies.

1 Introduction

Language models are now asked questions about living systems that used to go to a textbook or a colleague: what a channel blocker does to a cell's resting potential, why a tissue depolarizes, which intervention might restore a pattern (Singhal et al., 2023; Wang et al., 2023). The standard way to build such a model is to continue pretraining a general model on biomedical text and then tune it on question–answer pairs (Gururangan et al., 2020; Luo et al., 2022; Chen et al., 2023; Wu et al., 2024). The standard way to test it is a literature-derived benchmark such as PubMedQA (Jin et al., 2019). Both halves of this recipe stay inside text. Training rewards the model for sounding like the literature, and evaluation rewards it for agreeing with the literature, which it may have seen (Magar and Schwartz, 2022; Sainz et al., 2023). No step asks whether a claim would hold in the tissue it describes.

For most of biology there is no way to ask that question cheaply. For bioelectricity there is. The voltage across cell membranes (V_{mem}), shaped by ion channels, pumps and gap junctions, carries patterning information during development and regeneration (Levin, 2021), and the BETSE simulator integrates the electrodiffusion equations for whole tissues of coupled cells (Pietak and Levin, 2016). Given bath concentrations and channel permeabilities, BETSE returns the steady-state V_{mem} that a tissue reaches. A model's statement that a given intervention depolarizes the tissue to $-21\,\text{mV}$ can therefore be checked against the physics, with no annotator involved.

This matters because reinforcement learning with verifiable rewards has become the main way to teach models skills that imitation cannot (Lambert et al., 2024; Shao et al., 2024; DeepSeek-AI, 2025). The verifiers in that line of work are mostly string checkers for mathematics and code. A simulator is a different kind of verifier. It does not compare an answer to a reference that someone wrote down; it computes the outcome. The question we pursue is what such a verifier buys in a small domain model, and what it does not.

We built the whole system from scratch, from tokenizer to reinforcement learning, and ran it on a single consumer GPU. This was a methodological choice rather than a display of effort. Because we produced every training token, contamination could be audited instead of assumed away; because we wrote every stage, each failure could be traced to the stage that caused it; and because the system is small, every claim here can be re-run in hours.

The paper makes four contributions. First, it describes a complete pretraining and post-training pipeline for a 40M-parameter cell-biology model whose final stage is trained against a tissue simulator, together with the engineering problems that came up at each stage and how they were resolved (Sections 4–5). Second, it shows that a simulator-verified score on a standard held-out split can overstate what a model has learned, and that a sealed off-grid exam exposes the gap: the model had learned a lookup table, and the fix was in the placement of training data, not in the learning algorithm (Section 8). Third, it reports a controlled study of adapting a pretrained open model to the same corpus, in which every loss of dialogue ability is traced to a measurable property of the training data (Section 9). Fourth, it argues from these results for a division of labour, in which the weights carry language, a retrieval library carries facts with citations, and the simulator carries physical consequences (Section 10).

2 Related work

Domain language models. Domain-specific pretraining from scratch was shown to beat general pretraining for biomedical understanding by PubMedBERT (Gu et al., 2021), and generative models followed: BioGPT (Luo et al., 2022) and BioMedLM (Bolton et al., 2024) trained on PubMed, while BloombergGPT (Wu et al., 2023) mixed domain and general data. The more common route continues pretraining an open general model (Gururangan et al., 2020), as in Meditron (Chen et al., 2023) and PMC-LLaMA (Wu et al., 2024), and then instruction-tunes it, often on answers written by a stronger model. Our Track A is a from-scratch model in the BioGPT tradition but far smaller, chosen by the compute-optimal ratio for our corpus (Hoffmann et al., 2022); Track B is the continued-pretraining route run on a small open base (Ben Allal et al., 2025).

Post-training and verifiable rewards. Instruction tuning (Wei et al., 2022) and preference learning from human feedback (Ouyang et al., 2022) turned pretrained models into assistants. Direct preference optimization removed the separate reward model (Rafailov et al., 2023), and group-relative policy optimization (GRPO) removed the value network (Shao et al., 2024). Training on rewards computed by a program, rather than learned from preferences, has driven recent gains in reasoning (Lambert et al., 2024; DeepSeek-AI, 2025), with verification also applied at the level of steps (Lightman et al., 2023). We implement SFT, DPO and GRPO ourselves and replace the program verifier with a physics simulator. The over-optimization we observed with DPO matches the reward hacking reported by Gao et al. (2023).

Evaluation and contamination. Benchmark overlap with pretraining data inflates scores (Magar and Schwartz, 2022; Sainz et al., 2023). Likelihood-based multiple-choice scoring is sensitive to the prior over answer strings (Zhao et al., 2021; Holtzman et al., 2021), which turned out to be central to our PubMedQA result. LAB-Bench (Laurent et al., 2024) tests research-style biology tasks and was out of scope for a 512-token text model.

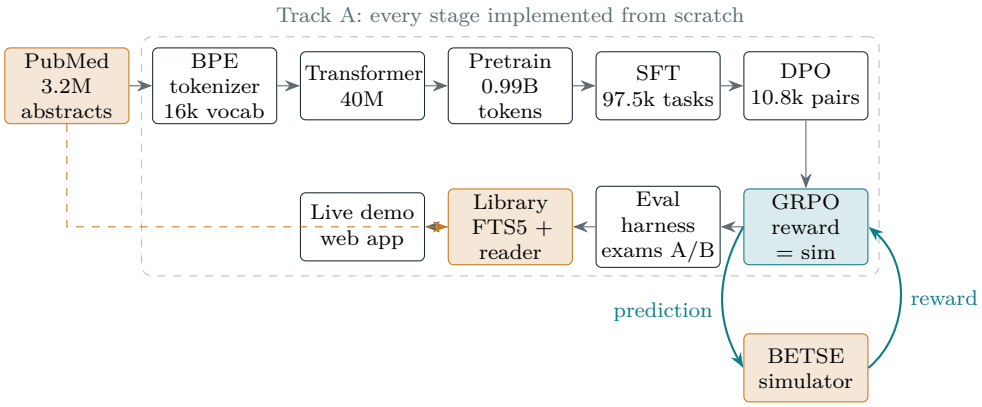


Figure 1: Pipeline. The amber components (corpus and library, simulator) tie the system to the domain; the rest is generic. Track B keeps the amber components and the exams and replaces the from-scratch transformer with an open pretrained model.

Forgetting and shortcuts. Sequential training on new data erodes earlier abilities (McCloskey and Cohen, 1989; Kirkpatrick et al., 2017), including in large language models under continual fine-tuning (Luo et al., 2023). Replaying earlier data mitigates this (Ibrahim et al., 2024), and LoRA (Hu et al., 2022) learns less and forgets less than full fine-tuning (Biderman et al., 2024). Models also latch onto incidental features that correlate with labels (Gururangan et al., 2018; Geirhos et al., 2020); we document two such cues that our own data mixture created. The view that post-training mostly teaches format (Zhou et al., 2023), and the finding that imitation transfers style more readily than capability (Gudibande et al., 2023), frame our Track B results.

Retrieval and long-tail facts. Small models recall rare facts poorly (Kandpal et al., 2023; Mallen et al., 2023), which motivates retrieval-augmented generation (Lewis et al., 2020). Our library uses classical full-text ranking (Robertson and Zaragoza, 2009) with a reader trained only on sentences mined from the corpus.

Language models in discovery loops. FunSearch (Romera-Paredes et al., 2024) and AlphaEvolve (Novikov et al., 2025) pair a language model that proposes candidates with an automatic evaluator that scores them, and quality-diversity search (Mouret and Clune, 2015) keeps the candidate pool varied. The simulator loop described here is the evaluator half of such a system for bioelectricity.

3 System overview

The system has a model in the middle and two components that tie it to the domain (Figure 1). The first is the corpus, 3.2 million PubMed abstracts, which serves as pretraining data and, through a full-text index, as a library the model can quote with PubMed identifiers (PMIDs). The second is the verifier: BETSE, which scores numeric predictions about tissue voltage. Everything between these two components is generic. Track A fills the middle with a model trained from random initialization; Track B fills it with an open pretrained model and keeps the corpus, verifier and exams fixed, so that the two tracks differ mainly in their starting weights. A web application, public at `https://drnamlabs.com/cellbio/`, runs the loop in front of the user: one specifies an intervention, the model predicts mean V_{mem}, BETSE runs, and the prediction is marked against the simulated tissue.

4 Building the model from scratch

All runs used seed 42 and a single RTX 5090 shared with other jobs, so every process was capped at 30 GB of GPU memory and 12 CPU threads. Checkpoints, logs, metrics and sample generations were written to disk at every stage, and failed runs were kept. Table 4 in the appendix lists the hyperparameters.

4.1 Corpus

We selected PubMed records under the MeSH headings Morphogenesis, Regeneration, Cell Physiological Phenomena and Tissue Engineering, with NCBI's own hierarchy expansion, keeping only records with an abstract. A validation pass on one baseline file gave a 5.0% in-domain yield and 516 tokens per document, which suggested a corpus of about 1.7B tokens. The full fetch ran into three undocumented behaviours of the E-utilities service. Retrieval through a stored result set stops after the first 10,000 records, so we split the query by entry date, bisecting any date range that still exceeded the cap, which gave 482 slices. The service sometimes returns HTTP 200 with truncated XML, so fetching and parsing were retried together, and a slice that kept failing was logged without stopping the run. Rate-limit replies come back as valid JSON without a result count, so a missing count was treated as a retry signal. The final corpus has 3,198,416 documents (1.6 GB compressed, no failed slices, no duplicate PMIDs). Its word count, 616M, corrected the earlier projection to roughly 0.8–1.0B tokens. Because the parameter count was chosen from the token count, this correction moved the model from a planned 80–100M parameters down to 40M. The five most recent slices are held out as a temporal validation set, so all validation text was published after all training text.

4.2 Tokenizer and architecture

The byte-pair encoder (Sennrich et al., 2016) was written from scratch and trained on the corpus with 16,384 tokens. It compresses the text to 4.91 bytes per token and keeps domain terms such as *depolarization* and *regeneration* as single tokens. Round-trip encoding is exact. The model is a decoder-only transformer (Vaswani et al., 2017; Radford et al., 2019) in plain PyTorch, without library transformer modules: 10 layers, 8 heads, width 512, context 512, rotary position embeddings (Su et al., 2024), RMSNorm (Zhang and Sennrich, 2019), SwiGLU feed-forward blocks (Shazeer, 2020) and tied input and output embeddings (Press and Wolf, 2017), for 40.5M parameters. Before any real training we checked the implementation three ways. A single batch was overfit from loss 9.77 to 0.049 in 68 steps. A hand-written attention agreed with PyTorch's fused kernel to 2.4×10^{-6}. Generation with the key–value cache matched full recomputation token for token.

4.3 Pretraining

The model was trained for one epoch over 988.5M tokens: 30,167 steps of 32,768 tokens with AdamW (Loshchilov and Hutter, 2019), a peak learning rate of 6×10^{-4} with 500 warmup steps and cosine decay, and bf16 autocast. The run took 62 minutes. We stopped at one epoch because the compute-optimal ratio was already met (Hoffmann et al., 2022) and repeated data gives diminishing returns (Muennighoff et al., 2023). Temporal-validation loss fell from 9.70 at initialization to 2.436 (perplexity 11.4), and training and validation loss stayed together throughout (Figure 2). Samples went from token soup to fluent methods prose about patch clamping and ATPase kinetics, with invented details.

4.4 Instruction tuning

We wanted the from-scratch track to contain no text written by another model, so instruction data was mined from the corpus. Four tasks were extracted from abstracts in the training slices:

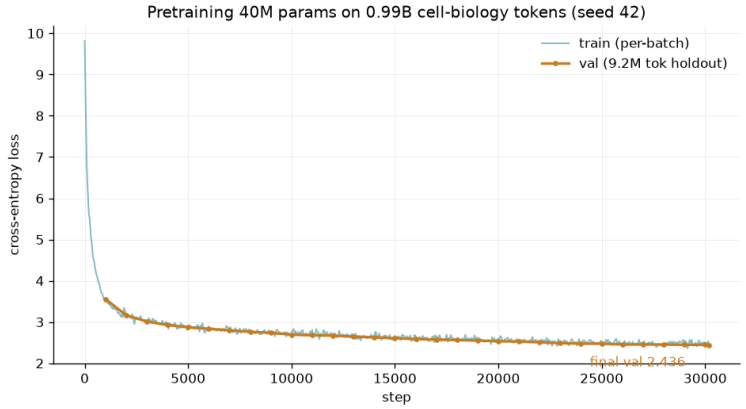


Figure 2: Pretraining loss of the 40M model on the training stream and the temporal hold-out.

titles phrased as questions paired with the abstract's conclusion, "X is a…" sentences turned into definitions, abbreviations paired with their expansions, and title generation. This gave 97.5k examples. Loss was computed on answer tokens only. Two epochs at learning rate 6×10^{-5} took about five minutes and brought masked validation loss to 2.316. The model switched from continuing text to answering it. One failure was systematic: the model answered "ATP stands for ATP". Common abbreviations are almost never spelled out in abstracts, so the abbreviation task never showed their expansions, and the model learned to copy.

4.5 Preference optimization

DPO (Rafailov et al., 2023) was implemented from the loss formula, using masked sequence log-probabilities under the policy and a frozen copy of the SFT model. At step zero the two models are identical and the loss must equal $\ln 2$, which it did. We mined 10,841 preference pairs without an external model. The chosen answer was the gold answer. The rejected answer was either the SFT model's own sample at temperature 0.8 or a circular hard negative of the form "X stands for X". The first run ($\beta = 0.1$, learning rate 5×10^{-6}, all tasks) eliminated circular answers but raised the four-gram repetition rate from 0.27 to 0.46, and answers drifted off topic. We traced this to two causes: a loose KL constraint, and noisy chosen answers in the question-title task, where the abstract's last sentence does not always answer the title. The second run kept only the definition and abbreviation tasks, raised β to 0.3 and lowered the learning rate to 2×10^{-6}. Circular answers fell from 4.4% to 0.0% on held-out prompts and repetition was unchanged (0.272 to 0.276). The ATP answer stopped being circular but was still wrong. Preference optimization can redistribute probability among answers the model can already produce; it cannot supply a fact the model never saw.

4.6 Reinforcement learning against an equation

Before using the simulator we tested the RL machinery on a verifier that could not be wrong: the Nernst equation, the single-ion law that BETSE integrates. A generator produced three families of problems: the numeric equilibrium potential in millivolts, the direction of change when a concentration moves, and the polarity of the resulting shift. These were split 85/15 by parameter combination, so held-out combinations never appeared in training. The reward parser gives full credit within 2 mV and partial credit within 10 and 25 mV; every gold answer must score 1.0, and a self-test enforces this. GRPO (Shao et al., 2024) was implemented with 16 prompts and 8 samples per step, advantages normalized within each group, a token-level policy-gradient loss and a KL penalty of 0.05 against the starting model. Two practical points decided whether it worked at all. Starting from the SFT model, most groups received the same low reward and gave no gradient; 400 steps of supervised warm start on the problem format fixed this. Groups in

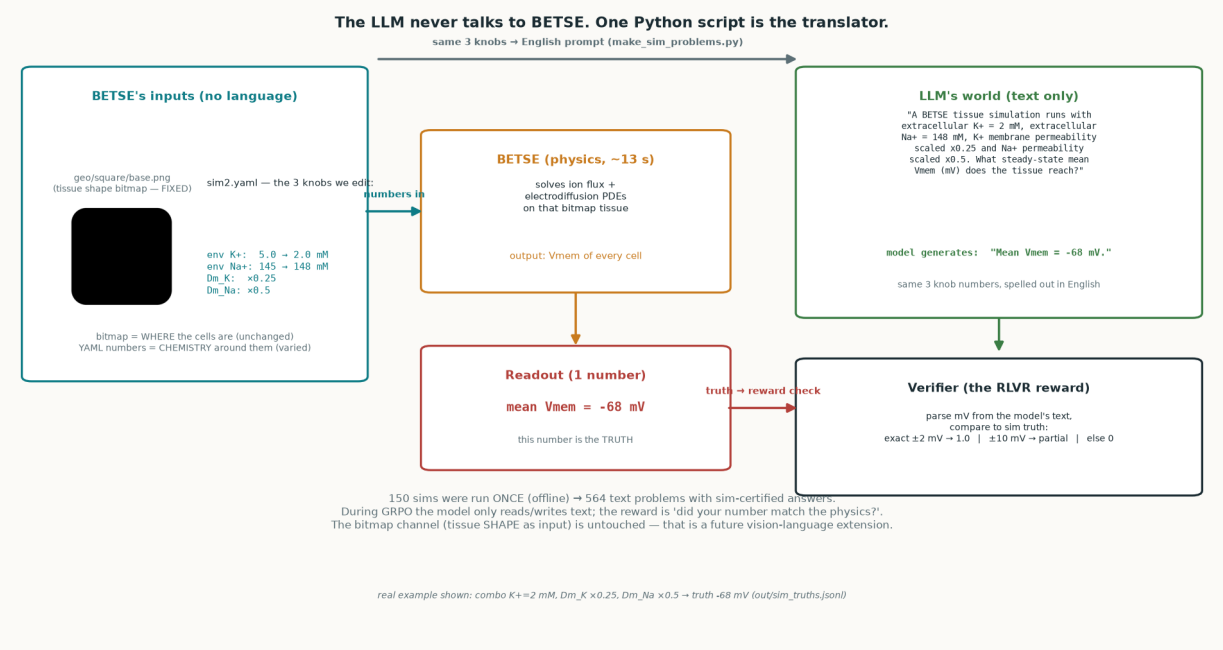


Figure 3: How the simulator becomes a reward. Interventions are simulated once and turned into text problems, and the model's numeric answer is parsed and scored against the simulated tissue.

which all samples received the same reward carry no signal and were skipped. Over 300 steps (7.5 minutes), training reward rose from 0.72 to 0.87. The categorical families generalized perfectly, but held-out numeric reward moved only from 0.565 to 0.599, while KL rose to 0.22. The model memorized the combinations it had been trained on and did not learn the logarithm.

5 The simulator as verifier

5.1 Running BETSE inside the training pipeline

BETSE 1.5.0 fails on Python 3.13, so it runs in a separate Python 3.11 environment and the training code calls it as a subprocess. Two settings caused errors that the simulator does not report clearly. Custom ion concentrations are silently ignored unless the ion profile is set to `custom`, and the intracellular and extracellular ion sets must each be electrically neutral or the run aborts. We therefore raised extracellular K^+ by lowering Na^+ by the same amount, as in a high-potassium Ringer's solution. On a 213-cell tissue, one configuration takes about 13 s from seeding to per-cell V_{mem} readout. As a first check, raising bath K^+ from 5 to 25 mM depolarized the tissue from -43.8 to -20.7 mV, as expected.

The language model never talks to BETSE during training (Figure 3). We ran 150 simulations offline, eight in parallel, over a grid of ten bath K^+ levels, five K^+ permeability multipliers (0.25–4) and three Na^+ permeability multipliers (0.5–2). This took 6.8 minutes with no failures. Mean V_{mem} spans -74.9 to -7.2 mV, and the slice at four times K^+ permeability is non-monotone in bath K^+, an effect of multi-cell coupling with no closed form. Each simulation became text problems that ask for the steady-state mean V_{mem} or the direction of the shift, giving 466 training and 98 held-out problems. The split was made by hashing the knob combination, so no held-out combination was ever simulated for training. The reward uses the same bands as the Nernst stage.

5.2 Training

Starting from the textbook model, 300 steps of supervised warm start on training gold answers were followed by 200 steps of GRPO with the same settings as before, with 60% of each batch

drawn from numeric problems.

5.3 A second exam, and a change of training data

The 98 held-out problems hide lattice *nodes*, not *regions*: every test point has trained neighbours one grid step away. To test generalization we built exam B from 60 new simulations at decimal knob values drawn log-uniformly inside the grid range, each at least 0.25 in $\log_2$ units (median 0.43) from every training node. This gave 234 problems, which were sealed before any model was scored on them. When the lattice-trained model failed this exam (Section 8), we asked whether the cause was the learning algorithm or the training data. We spent the same budget of 150 simulations on knob values drawn at random over the same range, each kept at least 0.2 $\log_2$ units from every exam-B point, and retrained with an identical recipe. A third variant added five nearest training examples, with their simulated V_{mem}, as text in the prompt, the same five-shot condition we gave the pretrained baselines. A first attempt at this failed: a model warm-started only on bare questions treated the examples as noise, and its accuracy fell when they were present. Warm-starting on a mixture with zero to five examples per prompt taught the model to use them.

6 Evaluation methodology

One harness scores every checkpoint on the same exams: perplexity on the temporal hold-out, PubMedQA (Jin et al., 2019), the 727 held-out textbook problems, and simulator exams A and B. PubMedQA is built from PubMed abstracts, so we checked it against our corpus: 59 of its 1,000 labelled items share a PMID with our training data. All PubMedQA scores are reported both on the full set and on the 941 clean items, and the two never differ by more than 1.1 points. Multiple-choice answers are scored by length-normalized log-probability of each option, which needs no answer parsing.

We compare against two kinds of reference. The first is a set of floors: chance, and a constant predictor that always gives the most common training answer for each problem type. The constant predictor scores 0.538 on exam A because the partial-credit bands reward a central guess, so it must always be reported. The second is a set of six open-weight chat models served locally, each given the same strictly formatted prompt with zero or five worked examples. One of them first scored 0.17. Reading its transcripts showed that it had written all of its output into a separate reasoning channel and left the answer field empty. After the harness was fixed to read the correct field, it scored 0.455 zero-shot and 0.664 five-shot. Since then we read raw transcripts before reporting any external model's score.

A second evaluation bug came from our own code. The library reader first scored a token-F1 of 0.39 on held-out questions. The batched evaluation had left-padded prompts, but the model had never seen padding and had no attention mask. Grouping prompts of equal length into the same batch, with no padding, raised the score to 0.990 with no change to the model. All batched generation in the project now groups prompts by length this way.

7 Retrieval library

The library is a SQLite FTS5 index over titles and abstracts of all 3.2M documents (8.2 GB, five minutes to build). Retrieval first requires all query terms and falls back to any of them, with a small synonym map and a rerank that favours sentences of the form "*term* is a *noun*". A reader was trained on 29,054 examples mined the same way as the SFT data, with a definition sentence as the answer and its surrounding abstract as context. In free generation the 40M reader stitched fragments together when the context lacked a clean definition. We therefore changed the mechanism from generation to selection. Candidate sentences are extracted from the retrieved abstracts, the reader ranks them by likelihood, and the best sentence is returned verbatim with its

	Pretrained	+SFT	+DPO	+GRPO (textbook)
Temporal-holdout perplexity ↓	11.50	14.59	14.59	16.24
PubMedQA accuracy (majority class 0.552)	0.537	0.491	0.496	0.460
clean subset (941 items)	0.526	0.480	0.486	0.453
Textbook physics reward (727 held-out)	0.000	0.239	0.238	0.977

Table 1: The Track A ladder on three exams it was not trained on.

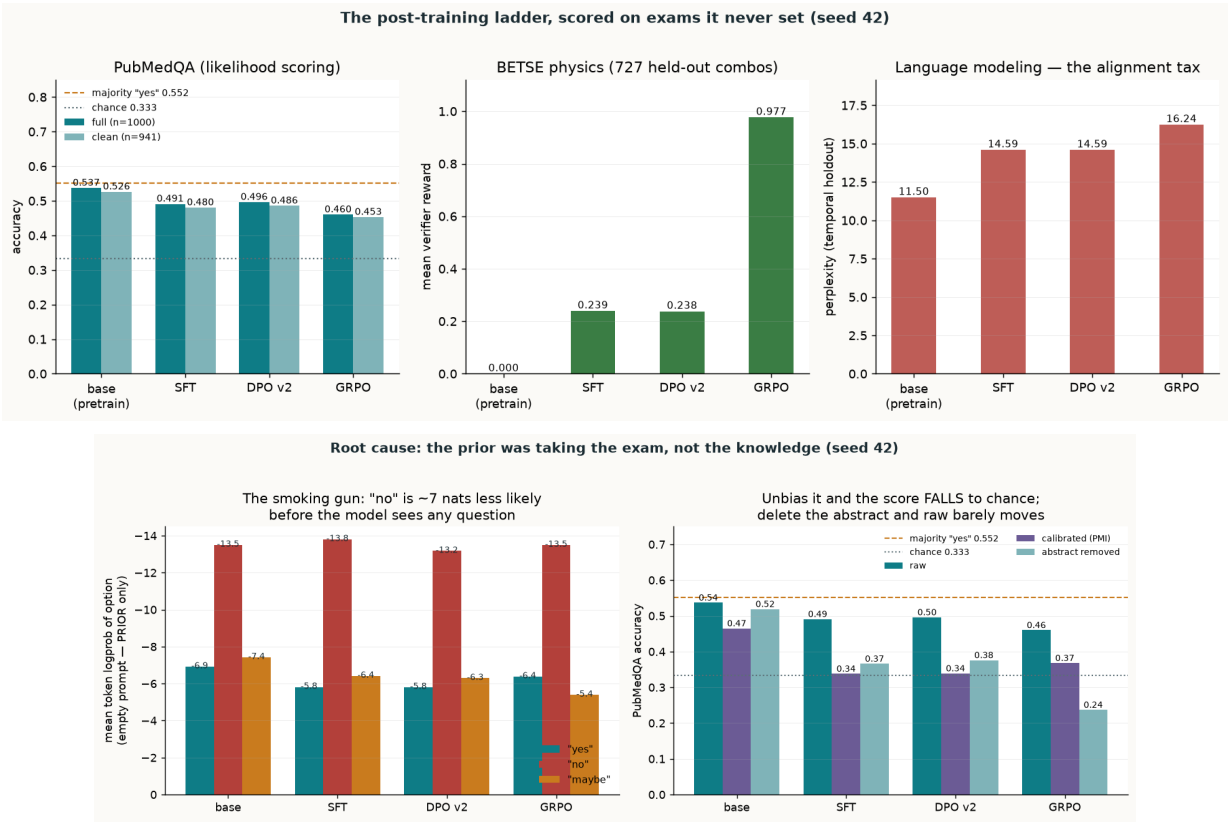


Figure 4: Top: the ladder on each exam. Bottom: the PubMedQA diagnosis. An empty prompt already prefers "yes" by about 7 nats, and removing the abstract barely changes accuracy.

PMID. If no candidate exists, the library declines to answer. Questions that are not definitional are declined outright, because an off-topic quotation proved worse than no answer.

8 Results

The training ladder. Each post-training stage bought a specific behaviour and paid for it in perplexity (Table 1). The textbook RL stage took held-out physics reward from 0.24 to 0.977, while perplexity on future abstracts rose from 11.5 to 16.2. On PubMedQA every checkpoint scored below the majority class. All four checkpoints chose "no" on zero of 1,000 items, which led us to test three explanations (Figure 4). Tokenization was ruled out. With an empty prompt, the model assigned "no" about 7 nats less log-probability than "yes" (−13.5 against −6.4). No training stage had ever produced a yes/no answer, so the score measured that prior, and the prior happened to mimic the majority class. Calibrating by the empty-prompt score (Zhao et al., 2021) allowed the model to choose "no", but accuracy fell to chance (0.34–0.47), and removing the abstract entirely moved raw accuracy only from 0.537 to 0.519. The model was not reading the evidence. We report the 0.46–0.54 range as an artifact of the prior, not as partial competence.

	Reward	Numeric only
Exam A: 98 held-out lattice combinations		
Constant predictor	0.538	0.454
Own ladder, zero-shot (best checkpoint)	0.321	0.150
Best open chat model, zero-shot	0.529	0.216
Best open chat model, five worked examples	0.904	0.852
Hosted model as agent, zero-shot	0.562	0.462
Hosted model as agent, five worked examples	0.955	0.952
40M, simulator warm start	0.938	0.898
40M, simulator warm start + GRPO	**0.945**	**0.912**
Exam B: 234 sealed off-grid problems		
Constant predictor	0.663	0.559
40M trained on the lattice	0.539	0.535
40M trained on 150 random points, same budget	0.777	0.657
+ five nearest worked examples in the prompt	**0.835**	**0.711**
Hosted model as agent, zero-shot	0.591	0.453
Hosted model as agent, five worked examples	0.808	0.742

Table 2: Simulator exams. All rows use the same verifier and the same reward bands.

Simulator exams. On exam A the simulator-trained model scored 0.945, and 0.912 on numeric problems alone (Table 2). Every generic checkpoint of our own ladder scored zero, because none could produce the answer format; the textbook RL model reached 0.321. The strongest open chat model scored 0.529 zero-shot, below the constant predictor, and 0.904 with five worked examples. The supervised warm start did most of the work (0.938), and GRPO added a little on top.

A hosted model as an agent. We also had a hosted model sit the exams as an agent (Claude Sonnet 5.5, closed book). The agent read one question file and wrote one answer file; it could reason before answering but had no access to the simulator or the truth files, and the same verifier scored its answers. In a first single test it scored 0.562 on exam A, close to the constant predictor, and 0.591 on exam B, below it. With the five nearest worked examples it scored 0.955 on exam A, level with the 40M model (0.945; 98 problems, so the gap is one problem) and 0.808 on exam B, below the 40M model's 0.835. Four of those ten batches had to be repeated because the agent reported using code, against the rules; the discarded attempts scored within 0.03 of the repeats.

That test could not say whether the gaps were real, so we built exam C before running anything further: twenty batches of fresh simulations (300 simulations, 1,146 problems), at points at least 0.25, 0.20 and 0.15 $\log_2$ units from every lattice node, random-training point and exam B point. The plan and the SHA-256 hash of each sealed file were published before any model or agent saw a batch. Each batch is one unit, so comparisons are paired over 20 batches (Table 3). The comparison fixed in advance was the 40M model with five examples against the hosted agent with five examples. The agent was ahead by 0.045 (95% bootstrap interval over batches 0.025 to 0.065, Wilcoxon $p = 0.0014$) and won 17 of 20 batches, so the near-tie of the first test did not hold. With five examples the agent also beat nearest-neighbour lookup (+0.025, $p = 0.003$), which the 40M model did not (–0.020, $p = 0.015$). Zero-shot the order reverses: the agent scored 0.106 below the constant predictor and the 40M model trained on random points 0.063 above it. We read that as the weights holding part of the simulator's response surface while the agent needs the examples to reach it.

A second round added a Haiku-class agent (Haiku 5.5) and four open models run through Ollama with greedy decoding and one-line answers (gpt-oss:20b, Qwen 3.5, Qwen 3.6 and DeepSeek-R1 8B). Their systems and comparisons were declared before they ran, and the plan records that the zero-shot column was made the headline after the first-round results were seen. Zero-shot, the 40M model trained on random points (0.772) is ahead of every hosted and open model, and

System	Zero-shot	Five examples
Claude Sonnet 5.5 (agent)	0.602	**0.852**
Claude Haiku 5.5 (agent)	0.569	0.842
40M, trained on random points	**0.772**	–
40M, mixed warm start	0.678	0.807
40M, trained on the grid	0.500	–
Qwen 3.5	0.465	0.784
Qwen 3.6	0.429	0.759
gpt-oss:20b	0.511	0.616
DeepSeek-R1 8B (truncated)	0.278	0.431
Constant predictor	0.708	0.708
Nearest-neighbour lookup (diagnostic)	–	0.827

Table 3: Exam C: mean reward over 20 sealed batches (1,146 problems). The 40M models were trained on this task; the others were not.

all of those are below the constant predictor. Given five examples, the large hosted models gain 0.25 to 0.27 and lead the table (Sonnet 0.852, Haiku 0.842; the two cannot be told apart, $p = 0.37$); the open models gain 0.10 to 0.33, with Qwen 3.5 (0.784) and Qwen 3.6 (0.759) above the constant predictor and gpt-oss:20b (0.616) and DeepSeek-R1 8B (0.431) below it. The 40M model with five examples is 0.035 below Haiku (Holm-adjusted $p = 0.02$) and 0.048 above Qwen 3.6 (Holm-adjusted $p = 0.04$). DeepSeek-R1 always reasons before answering and was given 3,000 tokens; 679 of its 1,146 zero-shot answers (59%) and 563 with examples (49%) were cut off before a final line and score zero, so its rows measure the token budget as much as the model. The 40M models were trained on this task and the hosted and open models were not, so the comparison is not like for like. Only the first comparison was the planned test; the rest are descriptive, with Holm adjustment over the whole secondary family. The limits are one run per batch, open models at default quantization, and tool use by the hosted agents that could be checked only through their own reports, none of which reported a breach on exam C. GPT and Gemini have not sat exam C, so it stays sealed.

Exam B changed the reading of these numbers (Figure 5). The lattice-trained model dropped to 0.539, below the constant predictor. Its predictions explained the failure: across all 234 problems it produced 17 distinct values, 16 of them answers it had memorized from training, and its predictions correlated with the true V_{mem} (0.52) barely more than with the answer at the nearest training node (0.47). It had learned a soft lookup table, and exam A, where every test point sits next to a trained node, could not tell the difference. Moving the same 150 simulations to random positions raised the off-grid score to 0.777. Predictions now tracked the truth (correlation 0.59) more closely than the nearest node (0.50), although the model still emitted only values it had seen in training. Five worked examples in the prompt raised the score to 0.835. GRPO on top of that model lowered the off-grid score to 0.812, and the mixed-example warm start cost zero-shot performance (0.666 against 0.777). Both costs come from the same recipe that produced the best score.

Library. On 946 held-out definition questions the reader reached token-F1 0.990, exact match 0.957, and 0.962 of answers were verbatim substrings of the retrieved source.

9 Adapting a pretrained model

Track A answers simulator problems well but cannot hold a conversation, write a paragraph or combine sources. Its context is 512 tokens and its training answers are single sentences. Track B asks whether starting from a model that already has those abilities, and adding the domain, gives both.

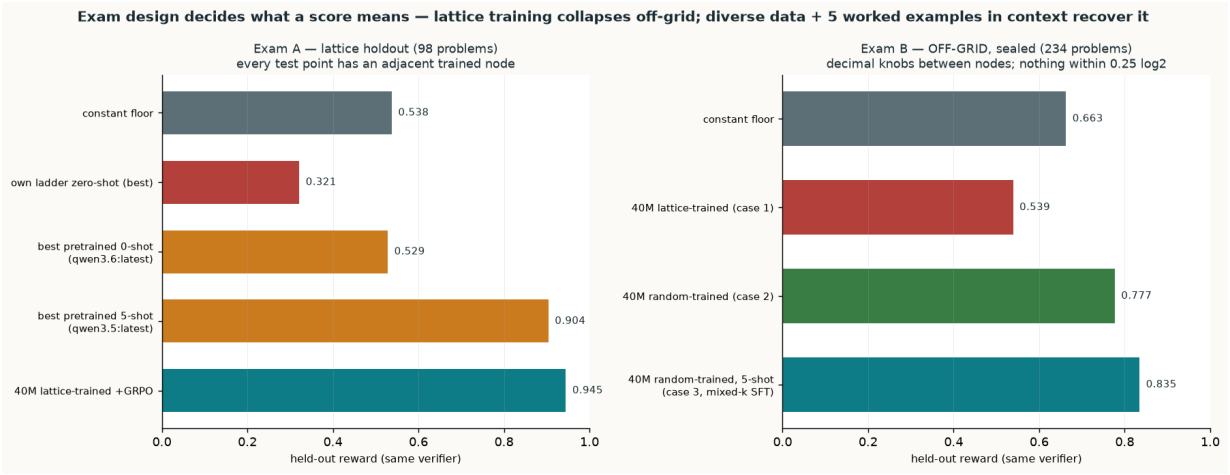


Figure 5: Exam A against the sealed off-grid exam B. The same model that leads exam A falls below the constant predictor on exam B; changing where the training simulations are placed recovers most of the loss.

9.1 Setup and engineering

We first tried Qwen2.5-0.5B (Qwen Team, 2024). Its 152k-token vocabulary made the output logits about 5 GB per micro-batch of eight sequences. This ran out of memory beside other jobs, so we used micro-batches of two with sixteen accumulation steps, which gave 16.5k tokens per second and an 18-hour epoch. Untouched, this model compressed future PubMed text slightly worse than our 40M model (0.714 against 0.698 bits per byte). We stopped the run and switched to SmolLM2-135M (Ben Allal et al., 2025), whose size is close to Track A's, so that the comparison changes mostly the starting point. Because the two tokenizers differ, per-token loss is not comparable, and we report bits per byte on the same temporal hold-out. The corpus took 1.06B tokens under SmolLM2's tokenizer. Full-parameter continued pretraining ran at 63.8k tokens per second, taking 4.6 hours for one epoch. For the LoRA route ($r = 64$ on all linear layers, 19.5M trainable parameters) throughput was 53.6k tokens per second. The GPU was 89% busy and the CPU 3%. Keeping the frozen base in bf16 gained only 2%. Doubling the micro-batch ran out of memory, because the 49k-vocabulary logits dominate activation memory, and graph compilation was not available on this platform.

9.2 Continued pretraining

Full-parameter continued pretraining took the base model from 0.825 to 0.640 bits per byte and passed Track A's 0.698 after about 131M tokens (Figure 6). Most of the gain came in the first quarter of the epoch. LoRA on the instruction-tuned model reached 0.758 after 65M tokens and 0.677 after the full epoch.

9.3 What happened to dialogue

Lower bits per byte did not produce a better assistant, and each failure traced back to the data. After continued pretraining the base model echoed questions instead of answering them, as Track A had before its SFT stage. SFT on Track A's 97.5k tasks taught the format but produced answers of 14–18 words. The SFT answers had a median length of 15 words, and the model learned when to stop from that distribution; the generation length limit is only a ceiling. LoRA did not preserve the instruct model's behaviour either. After one epoch, answers no longer ended on their own, slid into paper register ("In this article, we will discuss…"), ran into the title of an imagined next abstract and repeated phrases (Holtzman et al., 2020). LoRA forgot less than full fine-tuning (Biderman et al., 2024), but it still forgot.

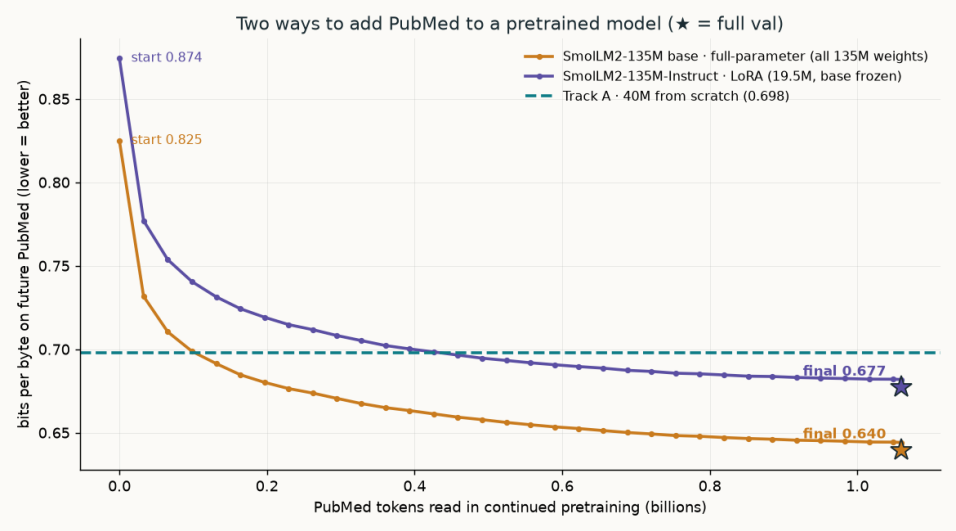


Figure 6: Held-out bits per byte during continued pretraining of SmolLM2-135M, full-parameter on the base model and LoRA on the instruct model, with Track A's from-scratch model as reference.

We then post-trained the continued-pretraining model on a mixture in the chat template: 80k multi-turn conversations from smol-smoltalk (Ben Allal et al., 2025), whose answers have a median of 187 words and were themselves written by larger models; 30k long domain examples mined from training abstracts; and 20k of Track A's short tasks. Loss was applied to every assistant turn. Self-termination went from 0% to 100% of probe answers, but domain answers stayed short. The cause was in our data. Our system prompt appeared on every domain example and on no general example, so it had become a mode switch. With it, the model gave 12–58-word answers in paper register. Without it, the model gave 173–196-word structured answers that were generic and wrong ("bioelectricity is electrical stimulation therapy"). We then drew the system prompt for every example from the same distribution: none, ours, or a generic one. The switch disappeared, and the topic took over its role. Biology questions got short paper-style answers with or without a system prompt (median 32 and 21 words), follow-up requests to explain more repeated the previous sentence, and one answer claimed that doubling extracellular K^+ doubles the membrane potential. In the mixture, biology questions were answered only in abstract style, and long conversational answers occurred only on other topics. No example showed a biology question answered as a long explanation, so the model had no way to learn that combination. These are shortcut features in the sense of Geirhos et al. (2020), created by how the mixture was assembled. Domain chat models usually get the missing examples from human writers or from a stronger teacher model (Ouyang et al., 2022; Mukherjee et al., 2023; Wu et al., 2024). The mixture also cost domain fit (0.644 to 0.695 bits per byte), the forgetting that replay is meant to prevent (Ibrahim et al., 2024).

Reference at 1.5B. Qwen2.5-1.5B-Instruct, untouched, scored below the constant predictor on both simulator exams (0.427 and 0.377). It wrote 150–170-word answers and followed three-turn dialogue, but stated an invented resting-potential equation with confidence. LoRA SFT on our own task data alone raised its scores to 0.914 and 0.839 with fluency intact. Its free-form explanations stayed unsourced and partly wrong. This model saw both the lattice and random training sets, so its exam-B score is not directly comparable with Track A's.

10 Discussion

The verifier made the small model competitive on the task it checks. A 40M model trained against the simulator outscored general models many times its size on the same numeric

problems. The advantage came from the training signal, not from size or cleverness. The same model, asked an open question, produces fluent abstract-style prose with invented details, and on PubMedQA its score reflects a prior, not reading. Simulator training made the model reliable on the problems the simulator scores and nowhere else.

The exam decides what the verifier certifies. Exam A used a standard held-out split and was scored by real physics, yet it rewarded a lookup table. Only an exam built to fall between training points showed this, and the remedy was a change in where training simulations were placed. For anyone building verifier-trained scientific models, this means that having a physical referee does not remove the need to design the exam. A held-out split that shares the training lattice measures memorization, however rigorous the referee.

Where knowledge should live. Both tracks gave confident wrong statements whenever a fact had to come from the weights, as reported for long-tail facts in much larger models (Kandpal et al., 2023; Mallen et al., 2023). Track B made this concrete. Continued pretraining improved compression of future abstracts by more than a fifth, yet the model still mis-stated basic electrophysiology, and every attempt to add dialogue on top moved the balance between register and content without fixing the content. We read these results as an argument for a division of labour. The weights should carry language: understanding a request, producing the format, reading a source. A citable library should carry facts, quoted with an identifier. The simulator should carry physical consequences, computed rather than recalled. Our system already has all three; what is missing is a model trained to move between them.

Limitations. Most results come from a single seed and are prototypes rather than defended claims. "Verified" means verified in BETSE, which is a model of tissue; agreement with it is not a biological result. The simulator problems vary three parameters, and the model fails on the sharp region of the response surface at high K^+ permeability, where the live demo shows misses of tens of millivolts. Track A cannot hold a dialogue or synthesize across sources, and its PubMedQA score is at the majority baseline. The Track B dialogue study used one small base model.

Toward action and discovery. The natural next step is for the model to output interventions rather than predictions: given a target voltage pattern described in words, it proposes knob settings, the simulator runs them, and the reward is the distance to the target. The same verifier then supports search. With the model as a proposal operator inside a quality-diversity loop and the simulator as the evaluator, in the manner of FunSearch (Romera-Paredes et al., 2024), the cost of a simulation, not the size of the model, becomes the limiting resource. We would count a result as discovery only if the loop finds a class of intervention that was not specified in advance. The architecture would stay the same if the referee changed from a simulator to an in-vitro assay; the model would remain decision support, with the final check made by an experiment.

11 Conclusion

We built a language model for cell biology from raw PubMed records to simulator-trained reinforcement learning, on one GPU, and measured each stage. A physics simulator turned out to be an effective reward: it made a 40M model better than much larger general models on the problems it checks. It also showed that a score is only as meaningful as the exam behind it, and that adding domain knowledge to a model's weights is a weak way to make it correct. The parts that worked were the simulator and the library that sit outside the weights, and the next version of the system will be built around them.

Reproducibility. All stages run on one consumer GPU with seed 42. Code, logs, metrics and figures are kept for every run, including the failed ones. The four Track A checkpoints, the tokenizer, a standalone generation script and a model card generated from the evaluation results are packaged for release.

Acknowledgements. BETSE was developed by A. Pietak and M. Levin. SmolLM2 and smol-smoltalk are released by Hugging Face. PubMed data are provided by the U.S. National Library of Medicine.

References

Loubna Ben Allal et al. SmolLM2: When smol goes big: Data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.

Dan Biderman et al. LoRA learns less and forgets less. *Transactions on Machine Learning Research*, 2024.

Elliot Bolton et al. BioMedLM: A 2.7B parameter language model trained on biomedical text. *arXiv preprint arXiv:2403.18421*, 2024.

Zeming Chen et al. MEDITRON-70B: Scaling medical pretraining for large language models. *arXiv preprint arXiv:2311.16079*, 2023.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning*, 2023.

Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020.

Yu Gu, Robert Tinn, Hao Cheng, Michael Lucas, Naoto Usuyama, Xiaodong Liu, Tristan Naumann, Jianfeng Gao, and Hoifung Poon. Domain-specific language model pretraining for biomedical natural language processing. *ACM Transactions on Computing for Healthcare*, 3(1), 2021.

Arnav Gudibande, Eric Wallace, Charlie Snell, Xinyang Geng, Hao Liu, Pieter Abbeel, Sergey Levine, and Dawn Song. The false promise of imitating proprietary LLMs. *arXiv preprint arXiv:2305.15717*, 2023.

Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In *Proceedings of NAACL-HLT*, 2018.

Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.

Jordan Hoffmann et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, 2022.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020.

Ari Holtzman, Peter West, Vered Shwartz, Yejin Choi, and Luke Zettlemoyer. Surface form competition: Why the highest probability answer isn't always right. In *Proceedings of EMNLP*, 2021.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Adam Ibrahim, Benjamin Thérien, Kshitij Gupta, Mats L. Richter, Quentin Anthony, Timothée Lesort, Eugene Belilovsky, and Irina Rish. Simple and scalable strategies to continually pre-train large language models. *Transactions on Machine Learning Research*, 2024.

Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William W. Cohen, and Xinghua Lu. PubMedQA: A dataset for biomedical research question answering. In *Proceedings of EMNLP-IJCNLP*, 2019.

Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. Large language models struggle to learn long-tail knowledge. In *International Conference on Machine Learning*, 2023.

James Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

Nathan Lambert et al. Tülu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

Jon M. Laurent et al. LAB-Bench: Measuring capabilities of language models for biology research. *arXiv preprint arXiv:2407.10362*, 2024.

Michael Levin. Bioelectric signaling: Reprogrammable circuits underlying embryogenesis, regeneration, and cancer. *Cell*, 184(8):1971–1989, 2021.

Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

Hunter Lightman et al. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Renqian Luo, Liai Sun, Yingce Xia, Tao Qin, Sheng Zhang, Hoifung Poon, and Tie-Yan Liu. BioGPT: Generative pre-trained transformer for biomedical text generation and mining. *Briefings in Bioinformatics*, 23(6), 2022.

Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *arXiv preprint arXiv:2308.08747*, 2023.

Inbal Magar and Roy Schwartz. Data contamination: From memorization to exploitation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.

Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.

Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.

Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.

Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems*, 2023.

Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. Orca: Progressive learning from complex explanation traces of GPT-4. *arXiv preprint arXiv:2306.02707*, 2023.

Alexander Novikov et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

Long Ouyang et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.

Alexis Pietak and Michael Levin. Exploring instructive physiological signaling with the bioelectric tissue simulation engine. *Frontiers in Bioengineering and Biotechnology*, 4:55, 2016.

Ofir Press and Lior Wolf. Using the output embedding to improve language models. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*, 2017.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

Bernardino Romera-Paredes et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016.

Zhihong Shao et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Karan Singhal et al. Large language models encode clinical knowledge. *Nature*, 620:172–180, 2023.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2024.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Hanchen Wang et al. Scientific discovery in the age of artificial intelligence. *Nature*, 620:47–60, 2023.

Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022.

Chaoyi Wu, Weixiong Lin, Xiaoman Zhang, Ya Zhang, Weidi Xie, and Yanfeng Wang. PMC-LLaMA: Toward building open-source language models for medicine. *Journal of the American Medical Informatics Association*, 31(9), 2024.

Shijie Wu et al. BloombergGPT: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, 2019.

Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. Calibrate before use: Improving few-shot performance of language models. In *International Conference on Machine Learning*, 2021.

Chunting Zhou et al. LIMA: Less is more for alignment. In *Advances in Neural Information Processing Systems*, 2023.

A Hyperparameters

Stage	Data	Optimization	Time
Pretraining	988.5M tokens, 1 epoch	AdamW, lr $6\times10^{-4} \rightarrow 6\times10^{-5}$, warmup 500, wd 0.1, 32.8k tok/step	62 min
SFT	97.5k mined tasks	lr 6×10^{-5}, 2 epochs, 3,048 steps, answer-only loss	5 min
DPO (v2)	10.8k pairs	$\beta = 0.3$, lr 2×10^{-6}, 438 steps	minutes
GRPO, textbook	4,028 train / 727 val	16 prompts $\times$ 8 samples, KL 0.05, 300 steps, lr 2×10^{-6}	7.5 min
GRPO, simulator	466 train / 98 val	warm start 300 steps, then 200 GRPO steps, 60% numeric	minutes
Library reader	29,054 mined QA	800 steps from the DPO model	minutes
Track B CPT, full	1.06B tokens	lr 10^{-4}, cosine to 10%, 32.8k tok/step	4.6 h
Track B CPT, LoRA	1.06B tokens	$r = 64$, $\alpha = 128$, lr 2×10^{-4}	5.5 h
Track B mixture	128.7k conversations	lr 10^{-4}, 1 epoch, 3,992 steps, max length 2048	47 min

Table 4: Training settings. Seed 42 throughout.